

Plan de ataque — Competencia de Programación con IA

Sistema de Registro y Monitoreo de Generación Solar por Departamento Ejecución con Claude Code · 11–12 de septiembre de 2026

0. Decisiones tomadas (no las reabras)

Cada decisión que se queda abierta te cuesta 20 minutos. Estas ya están cerradas:

Pieza	Decisión	Por qué
Framework	Laravel 13 (PHP 8.3+)	Obligatorio por bases
Panel admin / CRUD	Filament	Regala RF-01→RF-09 y RF-17 casi completos
Base de datos	PostgreSQL	Laravel Cloud y Railway lo dan gratis y ya provisionado
Mapa	Leaflet + OpenStreetMap	Sin API key, sin tarjeta, sin billing
Gráficas	Chart.js (vía widgets de Filament)	Ya viene integrado en Filament
API	Laravel <code>apiResource</code> + API Resources	Nativo
Doc API	<code>dedoc/scramble</code> (OpenAPI automático)	Genera la doc leyendo tus controladores
Deploy	Laravel Cloud (plan B: Railway)	Deploy nativo Laravel + Postgres incluido
Auth	Filament panel autenticado; dashboard público sin login	Evita fricción al evaluar

Si Filament 5 da problemas de compatibilidad en el momento de instalar, bajá a la mayor anterior sin discutir. No pierdas tiempo peleando versiones.

1. Regla de oro del día

Deploy vacío en la primera hora. El 80% de los equipos pierde la competencia por intentar desplegar a las 11 PM. Vos vas a tener la URL pública funcionando cuando la app todavía diga "Laravel". Después, cada fase termina con `git push` y deploy automático.

Segunda regla: **los datos de demo valen 4 criterios de la rúbrica a la vez** (dashboard, mapa, alertas, proyección). Un seeder pobre hunde los cuatro. Presupuestá tiempo real para esto, no lo dejes de último.

2. Esquema de base de datos (definido, no improvisado)

Esto es lo único que NO debe inventar la IA. Es la columna vertebral; si está mal, todo lo demás se tuerce.

```
departamentos
  id
  nombre
  codigo          string(3) unique      // GT-IZA, GT-GUA...
  cabecera        string nullable
  latitud          decimal(10,7)        // centroide, para centrar el mapa
  longitud         decimal(10,7)
  timestamps

modelos_panel          // RF-02
  id
  marca                string
  modelo               string
  potencia_kw          decimal(8,3)      // potencia nominal por unidad
  eficiencia            decimal(5,2) nullable
  estado               enum(activo, inactivo, descontinuado) default activo
  timestamps
  unique(marca, modelo)

granjas                // RF-03, RF-04, RF-07, RF-09
  id
  nombre
  departamento_id     FK -> departamentos
  municipio            string nullable
  direccion            string nullable
  latitud              decimal(10,7)
  longitud              decimal(10,7)
  familias_beneficiadas integer default 0
  generacion_esperada_mensual_kwh decimal(14,2) default 0
  fecha_instalacion    date nullable
  activa               boolean default true // desactivar, no borrar
  timestamps
  softDeletes

granja_panel           // RF-05 (pivot con datos)
  id
  granja_id            FK -> granjas (cascade)
  modelo_panel_id      FK -> modelos_panel
  cantidad              unsignedInteger
  fecha_instalacion    date nullable
  timestamps
  unique(granja_id, modelo_panel_id)

generaciones           // RF-08, RF-09
  id
  granja_id            FK -> granjas (cascade)
  periodo              date              // siempre día 1 del mes
  generacion_real_kwh   decimal(14,2)
  generacion_esperada_kwh decimal(14,2) // snapshot del período
  timestamps
  unique(granja_id, periodo)
  index(periodo)

alertas                // RF-14
```

```

id
granja_id          FK -> granjas
generacion_id      FK -> generaciones (cascade)
periodo            date
generacion_esperada_kwh decimal(14,2)
generacion_real_kwh decimal(14,2)
porcentaje_desviacion decimal(6,2) // negativo = por debajo
estado             enum(activa, revisada, resuelta) default activa
timestamps
index(estado)

```

Reglas de cálculo (van en `config/solar.php` , nunca hardcodeadas):

```

return [
    'factor_co2_kg_por_kwh' => 0.40,          // RF-10
    'umbral_alerta'         => 0.80,          // RF-14: real <= 80% de esperada
    'meses_historicos_proyeccion' => 12,
];

```

Valores derivados — atributos calculados, NUNCA columnas: - `capacidad_instalada_kw` = `SUM(granja_panel.cantidad × modelos_panel.potencia_kw)` (RF-06) - `co2_evitado_kg` = `generacion_acumulada_kwh × 0.40` (RF-10) - `porcentaje_desviacion` = `(real - esperada) / esperada × 100`

Si un evaluador pregunta "¿por qué no guardaste la capacidad en la tabla?" la respuesta es: *es un dato derivado, guardarlo introduce riesgo de desincronización cuando cambian los paneles*. Esa respuesta suma en "calidad de arquitectura".

3. Archivo `CLAUDE.md` (créalo ANTES de escribir código)

Esto es lo que hace que Claude Code trabaje bien todo el día en vez de irse por la tangente. Pegalo en la raíz del proyecto:

```

# Proyecto: Sistema de Generación Solar Guatemala

Competencia de 1 día. Laravel 13 + Filament + PostgreSQL + Leaflet.

## Reglas no negociables
- Idioma del dominio: ESPAÑOL. Tablas, modelos, campos y rutas en español (granjas, modelos_panel, generaciones, alertas). Código y comentarios en español.
- Nunca inventes API de Filament de memoria. Si no estás seguro de una firma, consultá la documentación oficial o revisá un archivo generado por el propio comando `make:` antes de escribir.
- Valores derivados = accessors o scopes. Nunca columnas duplicadas en BD.
- Constantes de negocio siempre desde config('solar.*'). Cero números mágicos.
- Ningún secreto en el código. Todo va a .env y .env.example.
- Después de cada cambio de migración: ejecutá `php artisan migrate:fresh --seed` y confirmá que corre sin error antes de decir que terminaste.
- No refactorices archivos que no te pedí tocar.
- Commits pequeños, mensaje en español, formato: `feat: ...` / `fix: ...`

## Antes de terminar una tarea
1. Ejecutá el código, no asumas que sirve.
2. Reportá qué probaste y qué salida obtuviste.

```

3. Si algo quedó a medias, decílo explícitamente. No lo escondas.

Contexto de negocio

- 22 departamentos de Guatemala.
- CO2 evitado = kWh × 0.40 kg.
- Alerta cuando generación real ≤ 80% de la esperada en el mismo período.
- Períodos son mensuales, almacenados como fecha del día 1 del mes.

4. Cronograma por fases

Cada fase = un contexto limpio en Claude Code (`/clear` entre fases) + un commit + un push. Usá **plan mode** (Shift+Tab dos veces) antes de arrancar cada fase: dejá que proponga el plan, corregilo, y recién ahí lo dejás ejecutar.

FASE 0 — Andamiaje y deploy vacío · 60 min

Objetivo: URL pública viva antes de escribir lógica.

```
composer create-project laravel/laravel generacion-solar
cd generacion-solar
composer require filament/filament
php artisan filament:install --panels
composer require dedoc/scramble
git init && git add . && git commit -m "chore: proyecto inicial"
# crear repo en GitHub y push
```

Luego, en Laravel Cloud (o Railway): 1. Conectar el repo de GitHub. 2. Provisionar PostgreSQL (se inyectan las variables solo). 3. Configurar comando de build/deploy: `php artisan migrate --force`. 4. Verificar que la URL carga.

No sigas a la Fase 1 hasta que la URL pública responda.

FASE 1 — Modelo de datos · 60 min

Prompt para Claude Code:

Creá las migraciones, modelos Eloquent y factories para este esquema exacto. No agregues campos que no estén listados ni cambies nombres. [pegar el esquema de la sección 2]

Además: - `config/solar.php` con `factor_co2_kg_por_kwh = 0.40`, `umbral_alerta = 0.80`, `meses_historicos_proyeccion = 12`. - Relaciones Eloquent en ambos sentidos, con tipado de retorno. - Casts apropiados (decimal, date, enum). - En el modelo Granja: accessor `capacidadInstaladaKw` que suma `cantidad × potencia_kw` de sus paneles, y accessor `co2EvitadoKg`. - Seeder `DepartamentoSeeder` con los 22 departamentos de Guatemala reales, con su cabecera y coordenadas del centroide. - Corré `php artisan migrate:fresh --seed` y mostrame la salida.

Verifícalo vos mismo: entrá a `php artisan tinker` y comprobá que los 22 departamentos están y que las coordenadas caen dentro de Guatemala (lat 13.7–17.8, lon -92.2 a -88.2). Es un error clásico que invente coordenadas de otro país.

FASE 2 — CRUDs en Filament · 90 min

Prompt:

Generá los recursos de Filament para `ModeloPanel`, `Granja` y `Generacion`.

- **ModeloPanel:** formulario con `marca`, `modelo`, `potencia_kw`, `eficiencia`, `estado`. Tabla con búsqueda por `marca/modelo` y filtro por `estado`.
- **Granja:** formulario con `nombre`, `departamento` (select con búsqueda), `municipio`, `latitud`, `longitud`, `familias_beneficiadas`, `generacion_esperada_mensual_kwh`, `fecha_instalacion`, `activa`. Tabla con columnas de `departamento`, `capacidad instalada calculada`, `familias`, y filtro por `departamento` y por `estado activo`. Acción de `desactivar` (no `borrar`).
- **Relation manager** en `Granja` para administrar sus paneles (`modelo de panel + cantidad`), mostrando la `capacidad total resultante`.
- **Generacion:** formulario con `granja`, `periodo` (selector de mes, se guarda como día 1), `generacion_real_kwh`, `generacion_esperada_kwh`. Al elegir `granja`, precargar la `esperada` desde `generacion_esperada_mensual_kwh`.

Validaciones (RF-17): campos obligatorios, numéricos no negativos, `latitud` entre -90 y 90, `longitud` entre -180 y 180, período único por `granja`, `cantidad de paneles` ≥ 1 . Mensajes de error en español.

FASE 3 — Alertas · 45 min

Prompt:

Implementá la regla de alertas (RF-14):

- `GeneracionObserver` que en `saved` evalúa: si `generacion_real_kwh <= generacion_esperada_kwh * config('solar.umbra1_alerta')`, crea o actualiza la `Alerta` correspondiente a esa generación; si deja de cumplirse, elimina la `alerta existente`.
- Guardá `esperada`, `real` y `porcentaje_desviacion` en la `alerta`.
- Recurso Filament de solo lectura para `Alertas`: `granja`, `departamento`, `período`, `esperada`, `real`, % de desviación con badge de color (rojo $< -40\%$, ámbar entre -20% y -40%), filtro por `estado` y por `departamento`.
- Comando artisan `solar:evaluar-alertas` que recorre todas las generaciones y reconstruye las alertas, para poder regenerarlas tras cargar datos masivos.
- Escribí un test que confirme: 79% dispara `alerta`, 80% dispara `alerta` (el requisito dice "al menos 20% por debajo", o sea $\leq 80\%$ es `alerta`), 81% no dispara.

Ese caso borde del 80% exacto es exactamente el tipo de cosa que un evaluador prueba. Tenelo resuelto y sabé defenderlo.

FASE 4 — Dashboard y reportes · 90 min

Prompt:

Construí el dashboard de Filament con estos widgets, todos alimentados por consultas agregadas (nada de cargar colecciones completas en memoria):

Stats nacionales: total de granjas, total de paneles instalados, capacidad instalada total en kW, generación acumulada en kWh, total de familias beneficiadas, CO2 evitado en kg y en toneladas.

Gráfica de barras: ranking de departamentos por generación acumulada (top 10).

Gráfica de líneas: generación real vs esperada por mes, a nivel nacional, últimos 12 meses.

Tabla de reporte por departamento (RF-12): una fila por departamento con cantidad de granjas, cantidad de paneles, capacidad instalada kW, generación acumulada kWh, familias beneficiadas y CO2 evitado kg. Ordenable por cualquier columna. Con filtro de rango de fechas que afecte las columnas de generación.

Widget de alertas activas: las 10 más recientes con enlace a la granja.

Todos los números formateados con separador de miles y las unidades visibles.

FASE 5 — Mapa interactivo · 60 min

Prompt:

Creá una página `/mapa` (pública, sin login) con Leaflet y tiles de OpenStreetMap:

- Centrada en Guatemala, zoom inicial que muestre todo el país.
- Un marcador por cada granja activa, con color distinto según departamento (paleta generada a partir del id del departamento, consistente entre recargas).
- Popup al hacer clic: nombre de la granja, departamento, municipio, capacidad instalada kW, generación acumulada kWh, familias beneficiadas, CO2 evitado kg, y badge si tiene alerta activa.
- Filtro lateral por departamento que oculta/muestra marcadores sin recargar.
- Leyenda de colores.
- Los datos se consumen desde `/api/v1/granjas/mapa` (GeoJSON o JSON plano), no embebidos en el Blade.
- Responsive: en móvil el filtro colapsa y el mapa ocupa el alto completo.
- Usá marker clustering si hay más de 30 marcadores.

Este es tu criterio. "Mapa interactivo y experiencia de usuario" es una línea entera de la rúbrica y es donde tenés ventaja sobre el resto. Invertí 20 minutos extra en que se vea pulido: tipografía decente, popups con jerarquía visual clara, estados hover. No lo dejes con el look default de Bootstrap.

FASE 6 — Proyección · 45 min

Método elegido: regresión lineal por mínimos cuadrados sobre los últimos 12 períodos.

Justificación para defender (memorizala):

Los datos de generación mensual tienen tendencia y ruido, con muy pocos puntos por granja. Una regresión lineal simple captura la tendencia sin sobreajustar, es determinista, no requiere entrenamiento ni dependencias externas, y cada coeficiente es explicable. Un modelo de ML con 12 puntos por granja sobreajustaría sin aportar precisión real. Cuando hay menos de 3 períodos, el sistema cae a promedio móvil, porque una recta con dos puntos no es una tendencia.

Prompt:

Creá App\Services\ProyeccionService con: - proyectar(Granja \$granja, int \$meses = 3): array — regresión lineal por mínimos cuadrados sobre los últimos `config('solar.meses_historicos_proyeccion')` períodos. Devuelve los meses proyectados con su valor, la pendiente, el intercepto y el R^2 . - Fallback a promedio móvil si hay menos de 3 períodos históricos; el resultado debe indicar qué método se usó. - Nunca devolver proyecciones negativas: piso en 0. - `compararProyeccionVsReal(Granja $granja)` — para períodos donde ya existe dato real, calcula qué habría proyectado el modelo con los datos previos y el error porcentual. - Widget en la ficha de granja: gráfica con histórico (línea sólida) + proyección a 3 meses (línea punteada), y una tabla de precisión histórica del modelo. - Tests con una serie perfectamente lineal (debe proyectar exacto) y con una serie plana.

FASE 7 — API REST + documentación · 45 min

Prompt:

Implementá la API REST versionada bajo `/api/v1` con API Resources:

- `GET /departamentos` y `GET /departamentos/{id}` (incluye estadísticas agregadas)
- `GET /granjas` (paginado, filtros: departamento_id, activa, búsqueda por nombre), `GET /granjas/{id}`
- `GET /granjas/mapa` — payload liviano para el mapa
- `GET /granjas/{id}/generaciones` (filtro por rango de períodos)
- `GET /granjas/{id}/proyeccion?meses=3`
- `GET /generaciones` (paginado, filtros por período y departamento)
- `GET /estadisticas/nacional` y `GET /estadisticas/departamentos`

- `GET /alertas` (filtro por estado y departamento)

Requisitos: respuestas con estructura consistente, códigos HTTP correctos, 404 con mensaje JSON legible, rate limiting, paginación con metadatos. Documentá todos los endpoints con anotaciones para Scramble y dejá la doc navegable en `/docs/api`.

Además generá `docs/API.md` con tabla de endpoints, parámetros y un ejemplo de respuesta por cada uno, por si la doc interactiva falla en producción.

FASE 8 — Datos de demostración · 45 min

No delegues los parámetros. Decíselos exactos:

Creá `DemoSeeder` que genere datos realistas: - 8 modelos de panel de marcas reales (Jinko, Trina, Canadian Solar, LONGi...) con potencias entre 0.35 y 0.7 kW. - 35 granjas distribuidas en al menos 15 de los 22 departamentos, con coordenadas reales dentro del departamento correspondiente (no aleatorias sobre el país). Nombres verosímiles tipo "Granja Solar Las Palmas", "Parque Solar Motagua". - Concentrá más granjas en departamentos de alta irradiación (Zacapa, El Progreso, Jutiapa, Santa Rosa, Izabal) para que el ranking cuente una historia. - Cada granja con 2 a 5 modelos de panel y cantidades entre 200 y 4000 unidades. - 18 meses de generación por granja, con: estacionalidad marcada (más generación en verano seco nov-abr, menos en época lluviosa may-oct), tendencia leve al alza, y ruido aleatorio de $\pm 8\%$. - **Exactamente 6 granjas deben quedar con bajo desempeño sostenido** (real entre 55% y 78% de la esperada en los últimos 3 meses) para que las alertas se vean pobladas. - 2 granjas con tendencia claramente creciente y 2 con tendencia decreciente, para que las proyecciones sean visualmente interesantes. - Familias beneficiadas proporcional a la capacidad instalada, entre 80 y 3500. - Al final corré `solar:evaluar-alertas` y mostrame cuántas alertas se generaron.

Después corré `migrate:fresh --seed` **en producción** y revisá el dashboard con ojos de evaluador.

FASE 9 — Pulido, documentación y presentación · 90 min

- `README.md`: descripción, stack, requisitos, instalación local paso a paso, comandos artisan propios, credenciales de acceso demo, URL pública, link al repo.
- `docs/USO-DE-IA.md`: entregable obligatorio. Qué herramienta usaste (Claude Code), en qué fases, qué revisaste y corregiste manualmente, y qué decisiones tomaste vos y no la IA (el esquema de datos, el método de proyección, el stack). **Este documento juega a tu favor: demuestra control, que es literalmente lo que pide la sección 15 del reto.**
- Revisar que `.env` no esté en el repo y que `.env.example` sí.
- Probar la app en un teléfono real. El requisito dice adaptable a distintos tamaños de pantalla.
- Presentación: 10–12 slides.

Guion de la presentación: 1. Problema y alcance (1 slide) 2. Arquitectura y modelo de datos (1 slide con el diagrama ER) 3. Demo en vivo: CRUD → dashboard → mapa → alerta → proyección (5–6 min) 4. Método de proyección y su justificación (1 slide) 5. API REST y documentación (1 slide) 6. Uso de IA: cómo la dirigiste y qué controlaste vos (1 slide) 7. Cierre: métricas del sistema

5. Cómo trabajar con Claude Code (esto define tu velocidad)

Hacé: - Plan mode antes de cada fase. Leé el plan, corregilo, después ejecutá. - `/clear` entre fases. Contexto sucio = alucinaciones. - Un commit por fase. Si algo explota, `git reset --hard` y volvé a intentar en vez de perder 40 minutos depurando código que no entendés. - Pedile que **ejecute y muestre la salida**. "Listo, ya funciona" sin output no es evidencia. - Cuando algo falle dos veces seguidas, parás y leés el código vos. La tercera iteración casi nunca la arregla.

No hagas: - No le des dos fases juntas. Una fase, un objetivo. - No aceptes que invente sintaxis de Filament de memoria. Es la fuente #1 de errores silenciosos. - No lo dejes tocar migraciones ya aplicadas en producción sin avisarte. - No pidas "hacelo bonito" sin referencia. Dale colores, tipografía y espaciado concretos.

Reparto entre los dos: Vos: Fases 0, 5, 8, 9 (deploy, mapa, datos de demo, presentación y pulido visual). Tu compañero: Fases 1, 2, 3, 7 (modelo, CRUDs, alertas, API). Fases 4 y 6 en conjunto. Trabajen en ramas separadas y mergeen al terminar cada fase, o coordinen para no tocar los mismos archivos al mismo tiempo.

Ojo con la regla 2 de la competencia: **ambos deben poder explicar la solución**. Reserven 20 minutos antes de la entrega para que cada uno le explique al otro la parte que no hizo. Van a preguntar.

6. Checklist final contra los requerimientos

Antes de entregar, verificá cada uno **en la URL de producción**, no en local:

- ☐ RF-01 — 22 departamentos cargados y asociables
- ☐ RF-02 — CRUD de modelos de panel con marca, modelo, potencia kW, estado
- ☐ RF-03 — Crear, editar, consultar y **desactivar** granjas
- ☐ RF-04 — Latitud y longitud almacenadas y visibles en el mapa
- ☐ RF-05 — Paneles asociados a granja con cantidad
- ☐ RF-06 — Capacidad instalada calculada automáticamente
- ☐ RF-07 — Familias beneficiadas por granja
- ☐ RF-08 — Generación real por período en kWh
- ☐ RF-09 — Generación esperada comparable con la real
- ☐ RF-10 — CO2 con factor 0.40 kg/kWh
- ☐ RF-11 — Dashboard con indicadores nacionales y por departamento
- ☐ RF-12 — Reporte por departamento con las 6 métricas mínimas
- ☐ RF-13 — Mapa con marcadores, selección e info de granja
- ☐ RF-14 — Alerta cuando real $\leq$ 80% de esperada
- ☐ RF-15 — Proyección implementada, documentada y justificada
- ☐ RF-16 — API REST con departamentos, granjas, generación y estadísticas
- ☐ RF-17 — Validaciones de obligatorios, rangos y relaciones

Entregables: - [] URL pública funcionando - [] Link de GitHub accesible - [] BD relacional (PostgreSQL) - [] README con instalación y ejecución - [] Documentación de la API - [] Documento de uso de IA - [] Datos de demo suficientes (dashboard poblado, mapa lleno, alertas visibles, proyecciones con forma) - [] Presentación lista

7. Plan de contingencia

Si a las 6 PM del día 12 vas atrasado, este es el orden de sacrificio:

1. **Primero se cae:** tests, comparación proyección-vs-real, clustering de marcadores, filtros avanzados.
2. **Después:** estilizado fino del mapa, gráfica de tendencia nacional.
3. **Nunca se cae:** deploy funcionando, mapa con marcadores, dashboard con los totales, alertas visibles, un valor de proyección en pantalla, API con 4 endpoints, datos de demo.

Una app desplegada al 80% gana contra una app perfecta en localhost. Siempre.